

Best Practice

User Guide

PeopleXD Insight

Table of Contents

Document Control	4
1. Overview.....	5
1.1. Purpose of the Guide	5
1.2. Prerequisites	5
1.3. Insight Overview	5
1.4. Report Management Best Practices	6
1.5. Master Views	7
2. Good Practice.....	8
2.1. Overview.....	8
2.2. Planning your Report.....	11
2.3. Building your Report.....	12
2.3.1. Adding Columns	12
2.3.2. Next Steps.....	13
3. Performance Considerations	14
3.1. Data and Views.....	14
3.1.1. Personnel Views.....	14
HR Person Master.....	14
Post Person.....	14
Appointment History.....	14
3.1.2. Payroll View	14
Pay History	14
3.1.3. WFM Views.....	15
Person Calc Hours.....	15

Table of Contents

4. Example Aggregation Reports.....	17
4.1. Overview.....	17
4.2. Tabular Example.....	18
4.3. Pivot Reports	20
4.3.1. Overview.....	20
4.3.2. Pivot Report Examples.....	21
Example 1: Counting Employees in Age Bands by Management Unit	21
Example 2: Summing Pay History by Analysis Type	21
Example 3: Summing Pay History by Groups of Pay Codes	24
Example 4: Sets of Pay Codes	25
5. FAQ and Avoiding Common Issues.....	26
5.1. How do I get the most recent record out of a view?.....	26
5.2. Error caused by dividing by zero	27
5.3. Filter Prompts and Calculation	27
5.4. Referencing calculation in other calculations	27
5.5. Pivots and Row Counts	27
5.6. Outer joins on reports	28
5.7. ORA-00923: FROM keyword not found where expects	28
5.8. Converting UDF fields to dates.....	28

Document Control

Version	Date	Author	Details
1.0	01/12/2023	Kat Murie	Published to Knowledge Base
1.1	08/05/2023	Kat Murie	Detail added to Section 2.1 on filters. Sections 5.7 and 5.8 added to FAQs.
1.2	03/12/2025	Pat Conlon	Added section 1.4 Report Management Best Practices . Updated section 1.2 Prerequisites with Insight user guide library link.

1. Overview

1.1. Purpose of the Guide

The purpose of this document is to outline various best practice principles to follow when writing a report for simplicity and better performance. This includes:

- Planning and building the report
- Performance considerations for data and views
- Aggregation reports
- FAQs and avoiding common issues

1.2. Prerequisites

This document is a best practice guide with an emphasis on report writing for optimal performance. Before reading this guide, it is beneficial to have:

1. A working knowledge of Insight and some experience or training.
2. Familiarity with Insight user guides, including the following. These can all be found in the [Insight user guide library](#) within the Help Centre.
 - Insight Fundamentals
 - Insight Key Concepts
 - Admin Area
 - Calculation Builder
 - Insight User Security
 - Useful Functions
 - Basic Calculations
 - iCues
 - Report Store
 - Interface Reports
3. Familiarity with creating tabular and pivot reports in Insight.

1.3. Insight Overview

Insight is PeopleXD's operational reporting tool. Insight utilises our XD BI layer, which is a collection of database views built from the PeopleXD tables. A business area is a collection of related

information that applies to a particular business process or set of processes. Access can be granted to users for different areas, for example pay user access.

XD Insight Folders, also known as views, refer to specific reporting blocks. Examples of views include:

- Personnel records
- Absence details
- Payment records
- Position History

A join is a relationship between two views, which allows you to combine data from more than one view in your report. Each business area contains at least one master table, which has many of the key attributes for the employee. These master views have joins to the other key views within that business area.

As you add columns to a report, the list of available views and columns dynamically filters to display the views that are available you based on what is already added to the report.

1.4. Report Management Best Practices

For important reports standard, iCue, and particularly interface reports that are critical to the business there are several practices you can utilise to prevent or mitigate against accidental editing or deletion of a report.

1. **Clone the report** - create a backup copy in Insight.
2. **Export the report** - save it to your share/hard drive, so you can import it again if need.
3. **Lock a report** - so only the owner can delete. Note: Only admins with an override menu option can edit or delete.
4. **Restrict Admin users** - Insight Admins can delete all reports, therefore the number of users with admin privileges should be kept to a minimum to avoid any accidental deletions.

Please see the Insight [Fundamentals](#) and [Admin](#) guides for further guidance on any of the above.

When building a complex report, you should apply some or all these practices, especially once you've finished and tested your report.

1.5. Master Views

Having columns from a master view ensures you have access to most views in that business area (some are standalone views). Most reports tend to centre on the relevant master views and include columns from other reachable views as required.

Unless necessary to a single business area master view, you can avoid unnecessary joins across business areas which can impact report performance. For example, some fields are available on both HR master and payroll master. If a report already utilises the HR person master, it would be more efficient to take additional fields from that view, rather than taking a field from the payroll master.

2. Good Practice

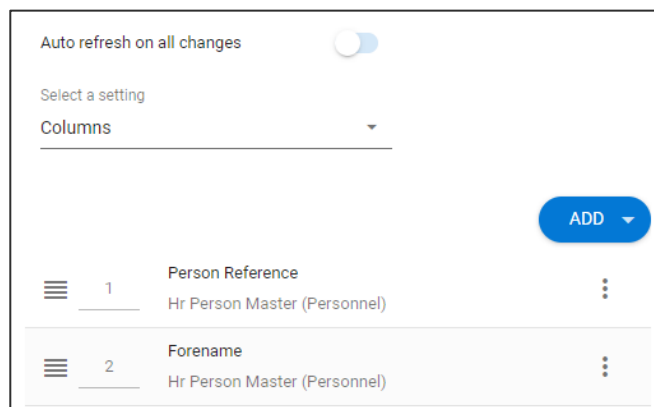
2.1. Overview

As an operational reporting tool using live data, it is the report writer's responsibility to create reports in an efficient manner so they can run in a reasonable time. Reports using the larger transactional views (for example appointment, pay history, calculate hours etc.) require particular attention.

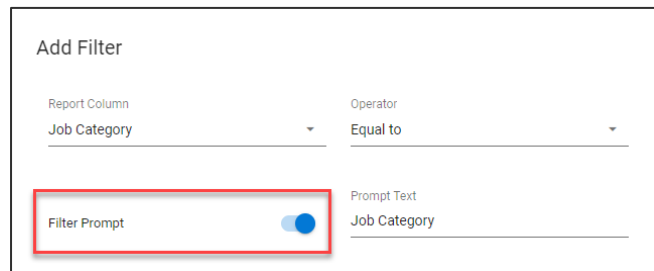
It is recommended to follow good practice guidelines:

1. Limit the number of columns and views used so the report only includes the required information.
 - a. Avoid adding unnecessary hidden columns, as they add to the report complexity as much as a visible column.
 - b. When editing a cloned report, remove any columns that are not necessary for the new report.
2. Filter up-front as much as possible to reduce processing time.
 - a. It is possible to filter not just on columns in your report, but any other columns from the views referenced in your report. Up-front filtering is the recommended method to ensure efficient performance.

The example below shows a report with two columns.



This can be filtered by all the HR Person Master columns available, even if they haven't been added to the report. For example, a job category filter from HR Person Master can be added, even though there is no job category column the report.



Add Filter

Report Column	Operator
Job Category	Equal to
Filter Prompt	Prompt Text
☒	Job Category

3. Minimise the use of functions/calculations as much as possible. Calculations can have a significant impact on report processing, so where possible, it is recommended to obtain data directly from the views.
4. Add parameters to the report using filter prompts to create a more focused report (for example, dates, pay groups, work groups, etc.).
5. Write calculations as efficiently as you can. For example, it would be inefficient to write a report with a pay history on all pay codes, and calculations to SUM pay codes you are interested based on case statements. Instead, it would be recommended to filter the paycodes in the report up-front and write simple calculations (sums without the case statement).
6. Avoid using calculations for filtering purposes (i.e. marking certain rows with an X or a 1 etc. for a certain condition and then filtering out all rows that don't meet the criteria). This is inefficient as every row is executed as part of the report query. Instead, it's recommended to use standard Insight filters, as this filtering is up-front and the full report query only executes one required data.

Example Calc 1:

CASE WHEN PRCI_PERSON_MASTER.PAY_GROUP in (100, 200, 300) THEN 'Y' ELSE 'N'

Instead of using a calculation like this and filtering on 'Y', it would be more efficient to use a standard Insight filter to filter the Pay Groups.

Add Filter

Report Column: Pay Group

Operator: In the list

Filter Text: 100,200,300 (11/3500)

Filter by Column: ☒

Filter Prompt: ☐

Ignore Case Sensitive: ☐ Active: ☒

CANCEL OK

Example Calc 2:

CASE WHEN PRCI_PERSON_MASTER.DATE_LEFT = PRCI_APPOINTMENT_HISTORY>ENDED.DATE
THEN 'Y' ELSE 'N'

Instead of using a calculation like this and filtering on 'Y', it's more efficient to use a standard insight filter to compare the two columns. The screenshot below displays an example of filtering by column. The column does not have to be in the actual report, it can be any of the columns within the scope of the views you are using in the report.

Add Filter

Report Column: Date Left

Operator: Equal to

Filter by Column: Ended Date

Filter by Column: ☒

Filter Prompt: ☐

2.2. Planning your Report

When planning a report, it is recommended to consider the following aspects:

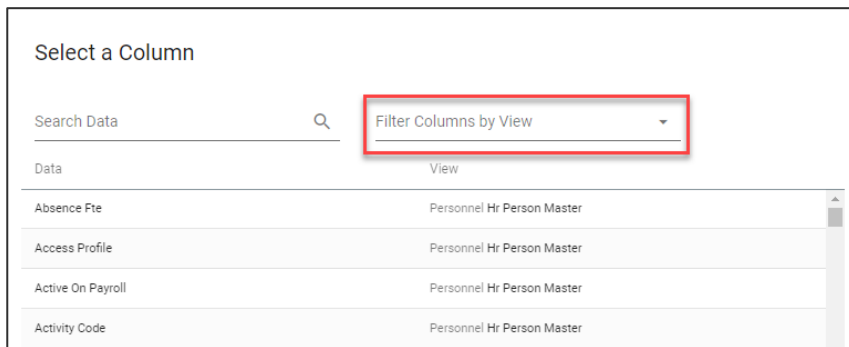
1. What areas the report will include. For example, it could focus on People Management, Payroll, WFM, or a combination of more than one area.
 - a. If you are unsure what Insight area should be included, consider what screens in PeopleXD contain the data you require. The screens will provide a good indication of the business area, and screen labels provide useful search terms for finding the required Insight columns.
2. What data is required. Consider whether the data you require is available on PeopleXD screen, or whether logic and calculations are required to get the desired output.
 - a. It is recommended to select a couple of test/reference employees and capture their details before building your report. This can be used to cross-reference against the data as validation.
3. What the dataset is. For example, it could focus on only permanent existing employees, or only staff from the finance department.

2.3. Building your Report

2.3.1. Adding Columns

Before any columns have been added to your report, all available views will be displayed in the dropdown list, but after adding columns, the list will filter down based on what is already added. Therefore, it's recommended to start a report with a master table (for example, by adding the person reference column) as they have joins to all the relevant views for a business area.

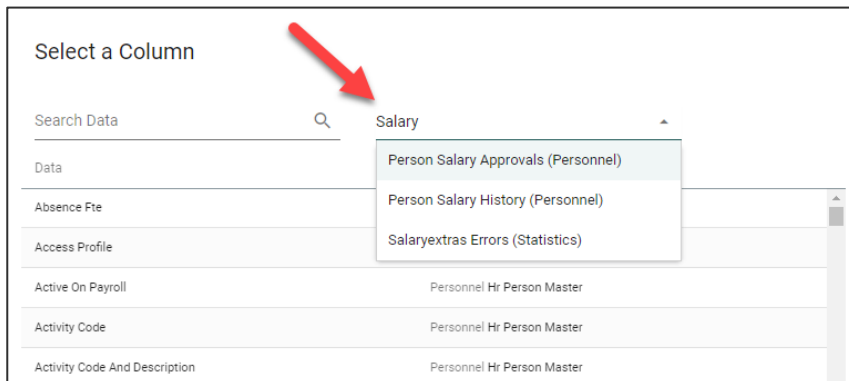
Start by selecting the relevant master view from the *Filter Columns by View* dropdown, then search for the required column. This will help to filter the views so that data is taken from the relevant master table.



Data	View
Absence Fte	Personnel Hr Person Master
Access Profile	Personnel Hr Person Master
Active On Payroll	Personnel Hr Person Master
Activity Code	Personnel Hr Person Master

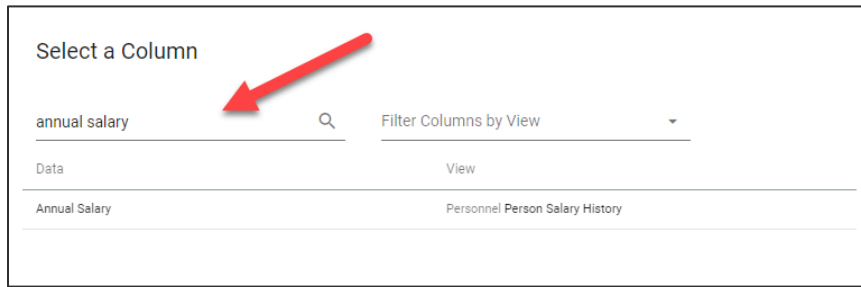
Note: When adding a column, pay attention to the name of the view, as a field can exist on multiple views.

If you are unsure of the required view, enter key search terms into the *Filter Columns By View* field to find relevant views (e.g. pay, history, absence, sickness etc.).



Data	View
Absence Fte	Personnel Hr Person Master
Access Profile	Personnel Hr Person Master
Active On Payroll	Personnel Hr Person Master
Activity Code	Personnel Hr Person Master
Activity Code And Description	Personnel Hr Person Master

Alternatively, if a specific column is required, enter key search terms into the *Search Data* field.



Select a Column

annual salary Filter Columns by View

Data	View
Annual Salary	Personnel Person Salary History

Once you have added a column from a particular view, it is recommended to filter by this view when adding additional columns. You can then browse all columns in the view, or search for a specific column using the search box.

2.3.2. Next Steps

1. Focus on building an initial report with straightforward fields first (e.g. master view type fields; person reference, names, department pay group, etc.).
2. Add in fields required from other views.
 - a. If unsure where you get the data, this is where it's useful to have the test employees data for reference from the PeopleXD screens, so you can compare any report data with what you see in the screens.
3. Add in any additional complex columns (for example, columns require function or logic).
4. Configure additional aspects, such as filters, sorts, formatting etc.

When satisfied with some sample records, expand your report to the full data set. It is recommended to use up-front filtering and parameters to focus the report as much as possible.

3. Performance Considerations

3.1. Data and Views

This section provides guidance on the larger transactional views in Insight to be aware of when designing a report.

3.1.1. Personnel Views

HR Person Master

The HR Person Master should be the starting point for personnel reports. This is the current information for each employee record in the system (active, dormant or future) relating to the Primary/Substantive contract exclusively. This view should be the starting point for most of your HR reports and has joins to all the other views in the HR area.

Post Person

Post Person contains data for current appointments and confirmed future appointments and should be used to return current appointment data in conjunction with HR Person Master.

Appointment History

Appointment History is the largest personnel view. It contains all changes and should only be used where you need retrospective info. This view is mainly used when the information you require is related to the appointment sequences, which cannot be obtained from the HR Person Master or Post Person.



Note: In some cases, the column you need may only be available on Appointment History, but it is recommended to check the HR Person Master and Post Person views first, then only add columns from Appointment History if required.

3.1.2. Payroll View

Pay History

When writing reports in Pay History, it is recommended to minimise the data being processed by filtering up-front (for example, by pay group, payroll periods, pay code, or analysis types). This will

make the report more efficient, and simplifies calculation writing (for example, by not requiring case statements with the list of pay codes when you do your summing).

When you are writing a report that is primarily focused on HR Person Master view, you may require some pay history elements (e.g. sick pay, overtime etc). In this scenario, it is recommended to avoid using any columns from pay history in your report. Instead, use the XD function *GET MUL CODE PERIOD VALUE* to return what you need.

This is particularly beneficial when the report deals with a large number of employees/history, as you are avoiding joins and parsing of a large set of pay history records, and using the function to return what you need. It can accept a list of up to 15 pay codes for the calculations. You can use multiple calculations/columns if you need to obtain different figures (e.g. sick pay, overtime etc). See below for an overview of the function:

GET MUL CODE PERIOD VALUE

Description: Get monetary value of one or more pay codes on a particular period.

Parameters:

p empno: Employee number

p from period: Period from

p to period: Period to

p pay code1: Pay code

...

p pay code15: Pay code

Output: Returns total monetary amount of the pay codes passed as parameters.

3.1.3. WFM Views

Person Calc Hours

When writing reports in Person Calc Hours, it is recommended to filter up-front (for example, by work group, pay code, calculated date from/to)

When you are writing a report that is primarily focused on HR Person Master view, you may require some elements from Person Calc Hours. In this scenario, it is recommended to avoid using any columns from Person Calc Hours in your report. Instead, use the XD function *GET PAY CODE HOURS* to

return what you need without referencing the view directly. See below for an overview of the function:

GET PAY CODE HOURS

Description: Return calculated hours based on dates and pay codes supplied.

Parameters:

p person: Employee number

p from date: From Date

p to date: To Date

p pay code1: Pay code

...

p pay code15: Pay code

Output: Return calculated hours for a list of pay codes (1..15) in a date period.

4. Example Aggregation Reports

4.1. Overview

Aggregation enables you to apply calculations to data in order to summarise the information (e.g. Sum, Count, Max etc).

In a tabular report you may want to use aggregation to group information about an employee into one row. Every column needs to be common, or else it cannot roll up to a single row. If this is not the case, the calculations for sums etc. will need to 'partition over' in your calculations, so that the value groups as required (e.g. person reference, or person reference, payroll period). See the practical examples below:

If the purpose of a report is to get the sum of all pay history for each employee across a set of pay codes and pay periods, you cannot have either pay code or pay period in the report columns. This is because there will be a number of different combinations of these, and the report will break them out to multiple rows.

Another example would be if a report focused on total absence days, you could not include either absence codes or reason codes on the report as they may differ across the data.

 **Note:** Hiding a column is not the same as omitting it from the reporting. A hidden column affects a report grouping the same way as if it was visible. The purpose of a hidden column is when it is required by the interim logic or another calculation, but it isn't necessary to display on the output.

The sections below provide some examples of aggregated tabular and pivot reports. As pivot reports are aggregation reports, they are generally the recommended method for building reports with sum, totals, sub totals etc.

4.2. Tabular Example

The screenshot below shows an example report focusing on the pay history, summing pay code by employee and analysis type:

Auto refresh on all changes ☐ Date as of 31-May-2023 12:24 (Record Count: 79)

Select a setting
Columns

ADD

	Forename And Surname	Person Reference	GrossPay	Deduction	ER
1	Forename And Surname				
2	Person Reference				
3	GrossPay				
4	Deduction				
5	ER				
	7274 MID PERIOD LEAVER PEADRC 4	2512246	1017.23	-411.97	1017.23
	Alan Sugar	2510918	27500	-11000	27500
	Stefan Brauer	7274000035	29919.35	-14018.28	29919.35
	Richie Lowry	7274000069	29919.35	-14018.28	29919.35
	Tom O'Neill	7274000008	49865.63	-24007.5	49865.63
	Francisco Gaspar	7274000031	29919.35	-14018.28	29919.35
	Sharon Looney	7274000005	69811.79	-34379.44	69811.79
	Tara Breslin	7274000045	17951.61	-7495.86	17951.61
	Mark O'Donoghue	7274000050	30000	-14057.4	30000
	Colman O'Reilly	7274000064	29919.35	-14018.28	29919.35
	Dean Forbes	7274000001	69811.79	-34379.44	69811.79
	7274 MID PERIOD STARTER PEADRC 4	2512232	1370.97	-555.24	1370.97
	Ken Murphy	7274000047	29919.35	-14018.28	29919.35
	Roy Kelly	7274000023	29919.35	-14018.28	29919.35
	Alan Burke	7274000067	29919.35	-14018.28	29919.35

The report is filtered by pay period (pay period is not included as a column in the report, it is only used as a filter):

Edit filter

Report Column
Payroll Period

Operator
Between

Filter Text
201925

Filter Text
201930

Because payroll period is filtered and grouping on a range of periods, the *Hide Duplicates* option in the report settings will need to be turned on. This ensures that we don't get row for each period. The screenshots below display the difference when this setting is on/off:

- Duplicate Rows disabled (results are grouped by the two pay periods on the example filter).

Forename And Surname	Person Reference	GrossPay	Deduction	ER
Barry Rice	7274000020	2500	-1142.13	2500
Barry Rice	7274000020	2500	-1142.13	2500

- Duplicate Rows enabled (this removes the grouping and displays one line as required).

Forename And Surname	Person Reference	GrossPay	Deduction	ER
Barry Rice	7274000020	2500	-1142.13	2500

To get a specific grouping for a calculated column, you will need to use a partition over employee to display a total value for the employee. In some instances you will need further partitioning to get the figure you require.

In this example, as the report has filtered by pay periods, the sum calculations below will need to partition over the person references. This ensures that the Sums are aggregated per employee and not employee, payroll_period.

Example Calculation Column (Gross Pay)

```
SUM(CASE WHEN PACI_PAY_HISTORY.ANALYSIS_TYPE = 'GP' THEN PACI_PAY_HISTORY.VALUE ELSE 0 END)  
over (PARTITION by PACI_PERSON_MASTER.PERSON_REFERENCE)
```

Example Calculation Column (Deductions)

```
SUM(CASE WHEN PACI_PAY_HISTORY.ANALYSIS_TYPE = 'DN' THEN PACI_PAY_HISTORY.VALUE ELSE 0 END)  
over (PARTITION by PACI_PERSON_MASTER.PERSON_REFERENCE)
```

Example Calculation Column (Employer)

```
SUM(CASE WHEN PACI_PAY_HISTORY.ANALYSIS_TYPE = 'ER' THEN PACI_PAY_HISTORY.VALUE ELSE 0 END)  
over (PARTITION by PACI_PERSON_MASTER.PERSON_REFERENCE)
```

Example Calculation Column (Nett Pay)

```
SUM(CASE WHEN PACI_PAY_HISTORY.ANALYSIS_TYPE = 'NP' THEN PACI_PAY_HISTORY.VALUE ELSE 0 END)  
over (PARTITION by PACI_PERSON_MASTER.PERSON_REFERENCE)
```

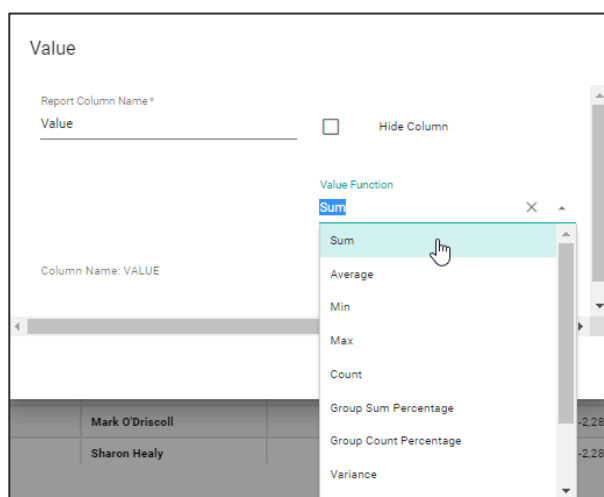
4.3. Pivot Reports

4.3.1. Overview

Pivot reports are by nature summary/aggregation reports and can be an easier way to summarise data. They can have better performance than large aggregation in tabular reports and require less complex calculations and no partition over logic.

Pivot tables consist of three elements:

1. Row: This can be thought of as the grouping for the line/row you want to display on the report.
2. Column: This is how you want the data summarised horizontally on the columns (e.g. departments, pay codes, absence type etc.) This is dynamic in nature as in you'll get a column for each department, paycode etc. found in the results set. This is also an advantage of using pivot tables.
3. Value: This is the data you are interested in aggregating. Expand the three-dot menu on this column to set the aggregation type (e.g. count, Sum, Average, Min etc.). Typical examples:
 - a. Count: Person References,
 - b. Sum: Pay Code Values
 - c. Average: Age



Note: The value function defaults to count, so ensure to update it correctly if another function is required, e.g. sum or max.

4.3.2. Pivot Report Examples

The following sections provide examples of pivot reports.

Example 1: Counting Employees in Age Bands by Management Unit

The below report counts person references by management unit, grouped by age range

Auto refresh on all changes ☐ Data as of 14-Jun-2023 10:42 (Record Count: 250)

Select a setting
Pivot

ACTIONS

Columns

Management Unit
Hr Person Master (Personnel)

Rows

Age Range
Hr Person Master (Personnel)

Values

Person Reference
Hr Person Master (Personnel)

Age Range	1000	1007	1023	Person Reference
Age 36-40	12.00	7.00		19.00
Age 56-60	3.00	1.00		4.00
Age 31-35	31.00	15.00	1.00	47.00
Age 61-65	2.00	1.00		3.00
Age 66-70	1.00			1.00
Age 46-50	3.00			3.00
Age 26-30	64.00	46.00		110.00
Age 41-45	6.00	3.00		9.00
Age 20-25	25.00	23.00		48.00
Age 51-55	4.00	2.00		6.00
Grand total	151.00	98.00	1.00	250.00

Rows: Age Range

Columns: Management Unit

Values: Person Reference (aggregation type set to count)

Example 2: Summing Pay History by Analysis Type

The below report sums pay code value, based on the pay history and the pay analysis types for each employee. This is equivalent to the tabular report example in section 4.2

Auto refresh on all changes ☐ Data as of 31-May-2023 12:25 (Record Count: 250)

Select a setting
X

ACTIONS

Columns

Analysis Type
Pay History (Pay)

Rows

Person Reference
Pay History (Pay)

Forename And Surname
Pay History (Pay)

Values

Value
Pay History (Pay)

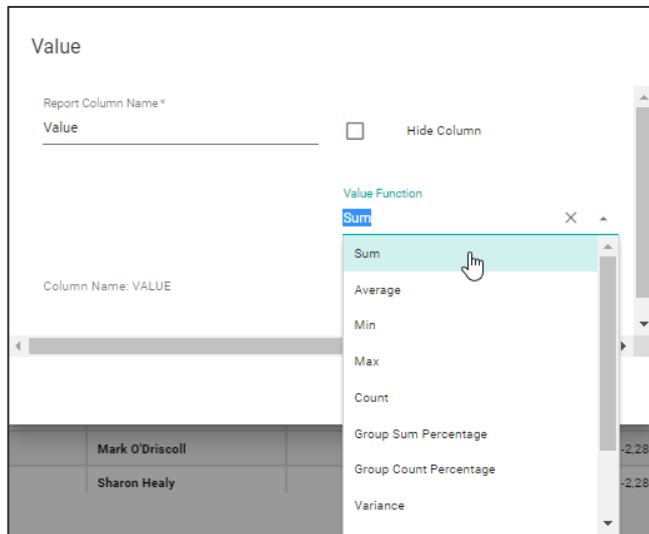
Person Reference	Forename And Surname	GP	DN	ER	NP	Value
7274000005	John Flood	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000052	Nathan O'Hehir	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000036	Thomas O'Neill	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000004	Sean Wiley	10,000.00	-4,650.67	-1,095.00	5,349.33	5,603.66
7274000055	Brian Cummins	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000061	Declan O'Connor	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000021	Ronan Loney	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000058	Rachel Foley	6,666.66	-2,951.16	-750.00	3,715.50	6,701.00
2511056	Period Daily Rate 3 Monday t Fri	5,000.00	-2,312.16	-547.50	2,087.84	4,828.18
7274000050	Mark O'Driscoll	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000053	Sharon Healy	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000037	Joseph Kearney	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000071	Samba Shiva	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
2512246	7274 MID PERIOD LEAVER PEAC	1,017.23	-411.97	-88.50	605.26	1,122.02
7274000018	Ed O Flynn	5,000.00	-2,384.26	-547.50	2,715.74	4,883.98
7274000013	Filipe Oliveira	2,500.00	-56.08			2,443.92
Grand total		110,183.89	-49,214.46	-11,768.50	58,525.51	107,726.44

Row: Person Reference, Forename and Surname

Columns: Analysis Type

Value: Pay History Value (set to aggregation type Sum)

Expanding the three-dot menu on the Pay History Value column allows you to choose the aggregation type.



Note: Unlike the tabular example, no calculations were required on the individual rows, as the pivot performed the categorisation and the summing of those values. Therefore, pivot tables are generally recommended as they can be easier to write for a non-experienced user.

Drill Down

Insight allows you to drill down to see the underlying data that is being aggregated for each pivot row. This can be useful to understand the underlying data behind the pivot and how a value has been calculated. Insight will generate the data when the report is run and summarise it in Pivot table. You can then double click a row to drill down and display the data used for that row. For example, this row displays the sum of an employee's pay history:

Person Reference	Forename And Surname	GP	DN	ER	NP	Value
7274000020	Barry Rice	5,000.00	-2,284.26	-547.50	2,715.74	4,883.98
Grand total		5,000.00	-2,284.26	-547.50	2,715.74	4,883.98

By double clicking on the row, the drill down screen opens and displays the two rows of data that were obtained based on you report logic, and summed in the pivot:

Drill Down					
Insights > TestpayhistorypivotPC > Drill Down					
Analysis Type	PAVROLL PERIOD	PAV_GROUP	Person Reference	Forename And Surname	Value
NP			7274000020	Barry Rice	1357.87
NP			7274000020	Barry Rice	1357.87

Pivot Total and Sub Totals

Totals and subtotals can be added to the pivot rows and columns by expanding the Actions menu and clicking Pivot Total:

Pivot Settings

Show column totals
☒

Show row totals
☒

Show column sub totals
☐

Show row sub totals
☐

CANCEL
OK

The screenshot below displays the example report with all totals turned off:

Person Reference	GP	DN	ER	NP
7274000033	2,500.00	-1,142.13	-273.75	1,357.87
7274000034	2,500.00	-1,142.13	-273.75	1,357.87
7274000035	2,500.00	-1,142.13	-273.75	1,357.87
7274000057		-1,086.05	-273.75	1,357.87
7274000058	3,333.33	-1,458.66	-365.00	1,874.67
7274000059	2,500.00	-1,142.13	-273.75	1,357.87
2510918	2,500.00	-1,000.00	-12.50	1,500.00
2510921	2,500.00	-1,156.08	-273.75	1,343.92
7274000036	2,500.00	-1,142.13	-273.75	1,357.87
7274000037	2,500.00	-1,142.13	-273.75	1,357.87
7274000060	2,500.00	-1,142.13	-273.75	1,357.87
7274000061	2,500.00	-1,142.13	-273.75	1,357.87
2511056	2,500.00	-1,156.08	-273.75	1,343.92
7274000001	5,833.33	-2,758.67	-638.75	3,074.66
7274000038	2,500.00	-1,142.13	-273.75	1,357.87
7274000039	1,500.00	-597.13	-130.50	902.87
7274000062	2,500.00	-1,142.13	-273.75	1,357.87

The screenshot below displays the example report with all totals turned on:

Person Reference	GP	DN	ER	NP	Value
7274000043	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000066	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000067	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000068	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000008	4,166.67	-1,892.00	-456.25	2,274.67	4,093.09
7274000009	4,166.67	-1,893.34	-456.25	2,333.33	4,210.41
7274000044	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000045	1,500.00	-597.13	-130.50	902.87	1,675.24
7274000069	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000070	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000010	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000011	3,333.33	-5.95	-365.00		2,962.38
7274000046	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000047	2,500.00	-1,142.13	-273.75	1,357.87	2,441.99
7274000048	2,500.00				2,500.00
7274000071	2,500.00	-56.08			2,443.92
Grand total	126,333.33	-54,919.23	-13,231.00	65,586.72	123,769.82

Example 3: Summing Pay History by Groups of Pay Codes

The following example is similar to example 2 but categorises pay code values by analysis type and period. The difference in this example is that calculations are used to derive the analysis types. This type of approach can be useful when only a specific set of columns is required (i.e. not every analysis type in the data).

Forename And Surname	Person Reference	Payroll Period	Gross Pay	Grand total Deduction	ER
Sean Carroll	7274000024	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Rory Rabak	7274000030	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Ruarri Oliver	7274000054	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Richard Cahill	7274000010	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Daniel Minney	7274000066	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Rory Gaggan	7274000014	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
John Flood	7274000065	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00
Nathan O'Neir	7274000052	201925	2,500.00	-1,142.13	2,500.00
		201930	2,500.00	-1,142.13	2,500.00

Row: Forename and Surname, Person Reference, Payroll period

Categories: None with this example (the values will drive it).

Values: Calculations have been used to sum based on analysis types (instead of using columns as in Example 2).

The screenshot below displays an example of a calculation for an analysis type = 'GP' column.

Calculation Name	Favourite
Gross Pay	☆

Data Fields	Calculated Inputs	XD Functions
Person Master (Pay)	Forename And Surname	
Pay History (Pay)	Person Reference	
Pay History (Pay)	Payroll Period	

Calculation Text

```
CASE WHEN PACL_PAY_HISTORY.ANALYSIS_TYPE = 'GP' THEN PACL_PAY_HISTORY.VALUE ELSE 0 END
```

Type here or select data/functions from the left

Note: If using multiple values in Pivots and some of these are calculations, ensure all values are calculation based and do not mix calculations and value functions (as used in Example 2). It's recommended to use one or the other, so the grouping of all the values will be consistent.

Example 4: Sets of Pay Codes

The below example is a pivot to summarise two sets of pay codes as columns. The approach used in Example 3 of using calculations in the values is the recommended way to achieve this.

Auto refresh on all changes ☐

Select a setting
Pivot

DATA as of 14-Jun-2023 09:45 (Record Count: 18)

Person Reference	Payroll Period	Grand total	
		Paycodes1	Paycodes2
7274000020	201904	1,444.19	6.00
	201908	1,513.95	6.00
	201912	1,513.96	6.00

Rows

- Person Reference
Person Master (Pay)
- Payroll Period
Pay History (Pay)

Values

- Paycodes1
Calculation
- Paycodes2
Calculation

Drill Down

The screenshot below displays the drill down displayed after double clicking on a row:

PAY_GROUP	Person Reference	Payroll Period	Paycodes1	Paycodes2
	7274000020	201904	2500	0
	7274000020	201904	0	-544.33
	7274000020	201904	0	-273.75
	7274000020	201904	0	-100
	7274000020	201904	-1055.81	0
	7274000020	201904	0	799.86

The below shows the calculations for the two value columns:

- Paycode1 Value Column**

```
CASE WHEN PACI_PAY_HISTORY.PAY_CODE IN (7500, 7561, 5646) THEN
PACI_PAY_HISTORY.VALUE ELSE 0 END
```

- Paycode2 Value**

```
CASE WHEN PACI_PAY_HISTORY.PAY_CODE IN (7564, 7571, 7589, 7591) THEN
PACI_PAY_HISTORY.VALUE ELSE 0 END
```

5. FAQ and Avoiding Common Issues

This section contains some general advice on avoiding common issues when building Insight reports.

5.1. How do I get the most recent record out of a view?

Some views (example training, leave, absences, etc.), have multiple records per employee and often you want to report on the most recent. A practical example of this would be if some employees have completed the same training course twice (therefore, there are two rows of data in the training log view for the same employee and course code), and you want to report on the latest records only.

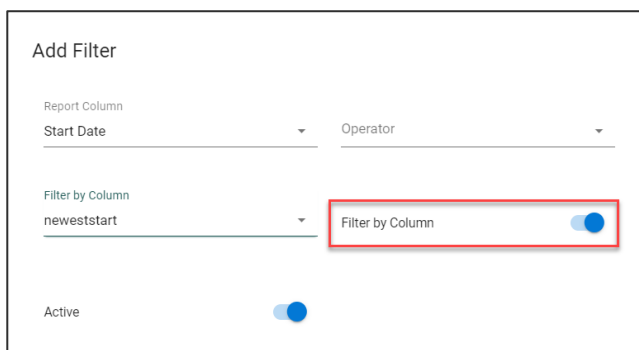
The basic approach is to use the training start date in the report and create a calculated column to return the max start date. Then, filter the report to only return rows where the start date column = max start date. See the steps below for further detail on this approach:

1. Create the report, making sure to include the required data columns (in this example, training record start date and course code).
2. Add a calculated column to get the max start date (in this example, it could be named *neweststartdate*). For example, the below calculation uses the max SQL function to return the latest start date based on employee, course code:

```
max (TRCI_TRAINING_LOG.START_DATE) over (PARTITION by
TRCI_TRAINING_LOG.PERSON_REFERENCE, TRCI_TRAINING_LOG.COURSE_CODE)
```

At this stage, multiple rows will still be returned for each employee.

3. To only return the latest record, use the *Filter by Column* toggle to only return rows where start date = neweststartdate



The screenshot shows the 'Add Filter' dialog box. It contains two main sections: 'Report Column' and 'Filter by Column'. In the 'Report Column' section, 'Start Date' is selected. In the 'Filter by Column' section, 'neweststart' is selected. A red box highlights the 'Filter by Column' toggle, which is currently turned on. At the bottom, there is an 'Active' toggle, which is also turned on.

5.2. Error caused by dividing by zero

When using division in a calculation, if the data ever results in the divisor being 0, this will result in an error during report execution. For example: *'ORA-01476: divisor is equal to zero'*

A simple division calculation can cause this error, for example:

```
CTCI_PERSON_BALANCES.CURRENT_BOOKED/CTCI_PERSON_BALANCES.CURRENT_BALANCE*100
```

To avoid this error, it is recommended to update the calculation, so it only attempts the division when the divisor is not 0. Therefore, the example above could be updated as follows:

```
CASE WHEN CTCI_PERSON_BALANCES.CURRENT_BALANCE !=  
0 THEN CTCI_PERSON_BALANCES.CURRENT_BOOKED/CTCI_PERSON_BALANCES.CURRENT_BALANCE*10  
0 ELSE 0 END
```

5.3. Filter Prompts and Calculation

When you use a filter prompt (e.g. date effective) and reference it in a calculation, this results in a dependency on the filter within the calculation. It is recommended not to change the name of the filter prompt or delete it without first removing it from the calculation, as this can cause errors. If you want to rename or replace the filter, ensure to add the filter prompt back into the calculation again.

5.4. Referencing calculation in other calculations

Referencing a calculation within another calculation introduces a dependency. For example, Calc A is referenced in Calc B, and you want to remove or rename Calc A. Best practice would be to remove the references to Calc A in Calc B, before making any amendments to Calc A. You can reintroduce the references as necessary, after making the amendments.

5.5. Pivots and Row Counts

The row settings in an Insight report allow you to set a max of 100,000 rows. For pivot reports, the max record count is the flat no. of data rows that are used to create the pivot table. This record count is displayed on screen when you run the report. The record count does not refer to the number of rows you are seeing in the pivot table; this will be less than the record count.

If you are writing a pivot table and hit 100,000 records, be aware that these are the only rows that will be used in your calculation. For example, if there are 110,000 rows the additional 10,000 will not be reflected, it is recommended to keep the record count under 100,000.

5.6. Outer joins on reports

For performance reasons, reports written in Insight by default do not use outer joins by default. This means if a report uses more than one view (e.g. Views A, B, C) a matching record will need to exist on each view, else that row will not return on the reports.

If necessary, outer join issues can be avoided by using a relevant function where available. This is also a useful approach if you're reporting from a view that does not have a join to the view that has the extra data required.

For example, if you want to include email address or mobile numbers, but not all employees have a record on Person Contact view, you can get this information using the XD function *Get Contact Text*. Using a function will return null if no record exists, but you will still get the line out in your report

Where an outer join is required for an important report, a case can be raised with support to request that outer joins are enabled for a that specific report.

5.7. ORA-00923: FROM keyword not found where expects

This error is usually caused by an error with one of your calculations. Open each calculation in turn and re-save them to review whether one of them shows an error. After identifying the incorrect calculations, update them to resolve the error.

5.8. Converting UDF fields to dates

User defined records are generic fields (based on text fields). When trying to filter a report on a UDF date, you will not get the date-based filter operator (e.g. between, last 7 days, month to date etc.). To filter, you need to add a calculation as shown below to convert fields (e.g. UDF record field2) to a date format. If you filter on the calculated column, you will then get the full range of date operators.

to_date (PRCI_PERSON_USER_DATA.TEXT2, 'DD/MM/YYYY')